KNN Classifiers: A From-Scratch Implementation Approach

39991938

2026-01-23

Abstract

Our project implements a K-Nearest Neighbours classifier from scratch in Python and a scikit-learn library where the main purpose is to have a comparison between both. All models were tested across multiple datasets, hyperparameter settings, and training conditions. Performance was evaluated using machine learning metrics and computational metrics, with statistical analysis conducted in R.

Results show that both implementations produce equivalent predictions with nearly identical accuracy. However, scikit-learn runs significantly faster due to its optimized data structures. Multiple Regression analysis was conducted for all performance metrics relative to data set features and implementation and dataset choice. Main results showed that implementation type is not a strong predictor on machine learning performance, but it is a strong influencer of computational efficiency. This further helps us defend our claims that both models are able to achieve similar levels of accuracy but one is substantially faster at performing this process.

Through series of experiments under diverse conditions we can demonstrate that building machine learning algorithms from scratch provides valuable educational insight and can achieve good levels of prediction as our implementation clearly shows. But for overall higher performance, especially better computational effectiveness, going with an existing implementation is our suggested choice.

Introduction

K-Nearest Neighbours is a foundational machine learning method used for prediction problems[IBM, 2024]. To decide the outcome of a new data point, this algorithm looks at the closest neighbours of that particular data point and whichever neighbour appears the most, results in the new value assigned as outcome[Zhang et al., 2016, James et al., 2013]. These models are classified as unparametric, meaning that they do not learn from a fixed mathematical function[Hastie et al., 2009]. Instead they are focused solely on the data by analyzing their behaviour, particularly trying to find similar points by looking at the ones closer to them.

KNN is also one of the earliest models discovered in ML resulting in many highly optimized libraries for this particular task[Halder et al., 2024, Scikit-learn Developers, 2024]. Using these libraries can be very powerful, although this paper will go through the rout of implementing one from scratch. We will be discussing in detail the differences of both approaches, and emphasizing the educational value that implementing something from the scratch provides.

The entire KNN theory was implemented from scratch in our custom implementation. Our KNN involves ability to predict through two distance metrics(Euclidean and Manhattan) and two voting schemes (uniform and distance-weighted) where predictions are performed on each to see the difference in performance throughout all these settings[Singh et al., 2017, Dudani, 1976]. Our sklearn implementation operates on identical conditions as well. Additionally, both models are applied on five datasets ranging from small(Iris, Wine) [UCI Machine Learning Repository, 1988, Aeberhard and Forina, 1992], medium(Digits, Breast Cancer)[Alpaydin and Kaynak, 1998, Wolberg et al., 1993] and large(Nursery)[Rajkovic, 1989]. Experiments are also carried on six k values(1,5,10,20,50,100) and three training splits(20%, 50%, 80%) equalling 336 cases for each model. Statistical analysis in R was performed[Wickham et al., 2019, Field

et al., 2012]. Wilcoxon signed-rank tests [Rainio et al., 2024], multiple regression[Krzywinski and Altman, 2015, Uyanık and Güler, 2013], decision boundary visualizations [Hastie et al., 2009, Olvera-López et al., 2005] and performance plots are used to analyze performance differences.

Our project's aims is to compare the overall performance of both models. We calculated machine learning and computational metrics across diverse hyperparameter conditions. Through detailed plots, regression analysis and decision boundary visualizations, we wanted to find out what drives performance in each model and how they differ from one another.

Methodology

The Model Algorithm

Our custom implementation was built completely from scratch in Python. It follows the core fundamentals of KNN theory based mainly on three functions: training, distance calculation and prediction[Zhang et al., 2016, Scikit-learn Developers, 2024, IBM, 2024, Halder et al., 2024, Cunningham and Delany, 2021, Abu Alfeilat et al., 2019].

During training, our model doesn't perform any calculations. It simply memorizes and stores the training samples and their respective feature names. Hence, these types of models spend most of the computational resources on the prediction phase[Uddin et al., 2022, Lopes and Ribeiro, 2023].

To perform predictions, our model first caclulates how far each point in the data set is to the outcome by using two types of distance measures, Euclidean and Manhattan distance. Euclidean distance works through measuring the distances between all points by summing the squared differences between each point relative to our outcome and taking the square root as the final result. Manhattan instead, sums the absolute differences between each point in the dataset to our outcome. Once we have all the distances, our model sorts them from largest to smallest based on the chosen K-value, followed by a voting process which decides the class of our outcome[Singh et al., 2017, Hu et al., 2016, Ehsani and Drabløs, 2020, Chomboon et al., 2015, Weinberger and Saul, 2009]. This voting occurs under two mechanisms: uniform voting and distance weighted voting. Under uniform voting each neighbouring point equals one vote, where the observations that occur mostly becomes the prediction of our outcome. In cases of ties, the first observation encountered among the neighbours is selected. Distance weighted voting on the other hand, operates similarly but it gives the closest observations more voting power, giving closer neighbours higher influence in being the final prediction[Dudani, 1976, Gou et al., 2011, Macleod et al., 1987, Samworth, 2012].

Our implementation also includes error handling to ensure inputs are valid. This includes rejecting non-positive k-values, verifying training data consistency, confirming the model is fitted before prediction, and ensuring k does not exceed the training set size.

To ensure our model runs correctly, unit tests were created and applied using the pytest framework[pytest Development Team, 2024]. These tests verified distance calculations using known values, confirmed that error handling properly rejects invalid inputs, checked that the fit method correctly stores training data, and validated that both voting schemes produce expected predictions on controlled examples. All seven tests passed successfully.

The scikit-learn implementation is assumed to follow the same algorithmic logic with more advanced optimization logic and perhaps more advanced KNN theory.These include efficient data structures such as ball trees and KD-trees that reduce neighbour search complexity, along with compiled Cython code that executes faster than interpreted Python[Scikit-learn Developers, 2024, Pedregosa et al., 2011].

Datasets

Five datasets were chosen to evaluate and test the performance of our models under diverse conditions, mainly trying to see the performance impact based on the nature and size of our data.

Table 1: Summary of datasets used in experiments

Dataset	Samples	Features	Output	Category
Iris	150	4	3	Small
Wine	178	13	3	Small
Breast Cancer Wisconsin	569	30	2	Medium
Digits (8x8)	1797	64	10	Medium
Nursery (UCI)	12960	8	5	Large

The Iris dataset contains measurements of sepal and petal dimensions for three flower species, representing a small and well-separated classification problem. The Wine dataset includes chemical analysis results for wines from three different cultivars. Both datasets are commonly used benchmarks due to their manageable size and clear class structure [UCI Machine Learning Repository, 1988]. The Digits dataset contains 8x8 pixel images of handwritten digits (0-9), presenting a higher-dimensional classification task with ten classes[Alpaydin and Kaynak, 1998]. The Breast Cancer Wisconsin dataset includes measurements from cell nuclei images for benign and malignant tumor classification, representing a binary medical diagnosis problem[Wolberg et al., 1993]. The Nursery dataset from the UCI Machine Learning Repository is the largest dataset used, containing categorical attributes related to nursery school evaluation[Rajkovic, 1989]. This dataset tests how both implementations scale to larger problem sizes [UCI Machine Learning Repository, 2024].

Experimental Design

To ensure fair comparison, both models were evaluated and ran under identical conditions. Two data pre-processing steps were applied for both models. First, each dataset was split into 3 training/test splits (20%/80%, 50%/50%, 80%/20%) and the second step applied was that all data we had was standardized usin z-score normalization and outcomes were encoded if neccessary(Nursery dataset). To ensure we have no data leakage, normalization metric were calculated on training data only and applied on training and test data respectively[James et al., 2013, Hastie et al., 2009].

In addition, our models operated and tested on 6 different k-values(1,5,10,20,50,100) to ensure that conditions are suitable regardless of the size of the data. That alongside different training splits, different distance metrics and voting types ensured that our design will produce 336 configurations per implementation, giving us space to thoroughly discuss how each model performs in different cases and how it compares with eachother.

Evaluation Metrics

The configurations produced were based on several computational and machine learning metrics. ML metrics involved were, accuracy (proportion of correct predictions), precision (proportion of positive predictions that were correct), recall (proportion of actual positives correctly identified), and F1 score (mean of precision and recall). All of them we're createed to assess the performance of our model throughout different hyperparameters[Sokolova and Lapalme, 2009, Powers, 2011].Computational metrics involved were fit time (duration of training phase), predict time (duration of classifying all test samples), and memory usage (storage required for training data, estimated using Python's sys.getsizeof function), and they were derived to assess the efficiency of our model compared to scikit-learn's implementation[Sivakumar et al., 2024, Demšar, 2006].

Statistical Analysis

Statistical analysis was done in R using mainly the tidyverse framework [Wickham et al., 2019].

Summary statistics were calculated by grouping results by implementation, dataset, k value, distance metric, voting scheme, and training size to obtain mean values for all metrics.

Wilcoxon signed-rank tests were used to see if the differences in machine learning and computational metrics between the two models were statistically significant at alpha = 0.05. This non-parametric test was chosen over the suggested t-tests due to our data not being normally distributed[Rainio et al., 2024, Demšar, 2006].

Seven multiple linear regression models were fitted to understand what factors had the highest influence on each performance metric. Each model used one metric (accuracy, precision, recall, F1, fit time, predict time, or memory) as the dependent variable, with implementation type, k value, distance metric, voting scheme, training proportion, and dataset characteristics as predictors. This allowed us to see how implementation choice affects performance, but also how other characteristics such as hyperparameter settings influence both classification accuracy and computational efficiency[Krzywinski and Altman, 2015, Uyanık and Güler, 2013, Hamilton et al., 2015, Shieh, 2020, Field et al., 2012].

Decision boundary plots were created to show how each model separates different classes. We used the Iris dataset with two features (petal length and petal width) and predicted class labels across a grid of points for both models at different k values. We chose the Iris dataset because its small size produces clear and easy to interpret boundaries. This lets us see if both models draw the same boundaries between classes[Hastie et al., 2009, Olvera-López et al., 2005].

Results

The following section demonstrates our results from the 673 experimental configurations of our modeles(336 per model) alongside the conclusion we draw from them. Through these experiments across multiple datasets and hyperparameter settings, we compared the overall model performance using machine learning and computational metrics. Then we used regression analysis and decision boundaries to uncover the detailed differences between implementations.

Overall Performance Comparison between Models

Table 1 presents the overall performance comparison between implementations:

Implementation	Accuracy	Precision	Recall	F1	Fit Time (s)	Predict Time (s)	Memory (KB)
custom	0.921	0.888	0.867	0.869	0.000001	20.42	212.8
sklearn	0.915	0.883	0.861	0.862	0.002131	0.04	212.8

Both implementations achieved almost the same prediction accuracy. Our custom model reached a 92.1% accuracy compared to scikit-learn's 91.5%. Precision, recall, and F1 scores were also compared, but differences were not that large compared to other performance aspects. Computational efficiency between models was substantially different. The custom implementation took an average of 20.4 seconds per prediction, where the main struggle was on larger datasets, while scikit-learn completed the same task in 0.04 seconds—approximately resulting in being 484 times faster. To ensure the differences between models are statistically significant, we used the Wilcoxon test. Compared to several types of statistical testing, this method requires no assumption about the data distribution, allowing for more accurate calculations in our case.

Table 2 shows the Wilcoxon signed-rank test results to determine if these differences are statistically significant:

Metric	W Statistic	p-value	Significant
Accuracy	6734	0	Yes
F1 Score	7570	0	Yes
Fit Time	0	0	Yes
Predict Time	56424	0	Yes
Memory	0	NaN	NA

The Wilcoxon tests conclude that both models are statistically different in accuracy, F1 score, fit time, and predict time(at alpha = 0.5) metrics. Although the accuracy difference between our implementations is concluded significant, we still believe that the difference is not high enough to produce any significant change in real world applications . When it comes to memory usage, our statistical calculation returns NaN as both implementations store the same training data, leading to the same memory needed for that process.

Detailed Comparison Relative to Hyperparameter Settings

For our experiments across several datasets, four identical hyperparameters settings were chosen for both models: training set size, number of neighbors (k), distance metric, and voting scheme. The following section will showcase the changes the models undergo throgh these conditions and the cases where they perform best. To help understand where our conclusions come from, this section also includes detailed visualizations that are provided in the Appendix (Figures 1-4).

Hyperparameters effect on Machine Learning Aspects

Both our custom and sklearn implementations achieved almost identical accuracy values and patterns accross all hyperparameter settings(see Figure 1, Appendix). Pertaining to training size, it's increase was always accompanied by accuracy increases as well (89% to 94% overall). These results also align with statistical theory that the more data experiments have, the better predictions there will be.

Setting k-value had a substantial impact on model performance. Deciding the number of neighbours significantly changed the likelyhood of achieving an accurate prediction. Best performance was detected between k=5 and k=10 values. As model approached k=100, performance dropped because the model considers too many neighbors and loses the ability to tell classes apart. At k=1, results were unstable because the model relies on just one point, which increases the chance substantially of the classification reading an outlier or noise rather then a true prediction.

Differences between distance metrics and voting scheme were found to be smaller. Throughout most datasets and implementations Manhattan distance was only slightly better then Euclidean distance and distance-weighted voting showed slight performance improvements over uniform voting.

F1 scores followed similar trends as accuracy (see Figure 2, Appendix). Results tend to have a greater positive impact with increased training sizes, better performance on lower k-values and overall small differences between distance and voting choices.

Hyperparameters effect on Computational Aspects

Computational time showed substantially larger differences between implementations (see Figure 3, Appendix). Computational time behaves differently depending on which hyperparameter we change. Training size is the only setting that clearly affects computation time - as training proportion increases, all lines move upward. This makes sense because more training data means more distance calculations.

The other three panels tell a different story. Changing k value, distance metric, or voting scheme produces almost flat lines. Whether we use 1 neighbor or 100, euclidean or manhattan, distance-weighted or uniform voting, computation time stays roughly the same. All of this information helps us conclude that the costly operation is distance calculation, not neighbor selection or voting.

Memory usage was identical between implementations (see Figure 4, Appendix). The solid and dashed lines overlap completely in the memory plots, confirming that both implementations store the same training data. Memory scaled only with training set size, while k value, distance metric, and voting scheme had no effect on memory consumption. This result was expected since KNN stores all training samples regardless of any hyperparameter settings.

Regression Analysis

To understand which factors mostly influenced model performance, multiple linear regression models were fitted for each outcome variable. Seven linear regressions were implemented: four for machine learning metrics (accuracy, precision, recall, F1 score) and three for computational metrics (fit time, predict time, memory usage). Each model included implementation type, hyperparameters (k, distance metric, voting scheme, training size), and dataset characteristics (number of samples, features, and classes) as predictors.

Predictors of Machine Learning Performance

Outcome	R^2	Implementation Significant?	Key Predictors
Accuracy	0.37	No	k, train size
Precision	0.67	No	k, train size
Recall	0.84	No	k, train size
F1 Score	0.79	No	k, train size
Predict Time	0.53	Yes	Implementation, n_samples
Fit Time	0.46	Yes	Implementation, n_samples
Memory	0.84	No	n_samples, train size

The regression analysis showed that implementation type does not affect any machine learning metric. Whether we use our custom model or scikit-learn, the predictions are the same. For accuracy, the model explained approximately 62% of variance (adjusted R^2 = 0.62). This means that 62% of the differences in accuracy scores can be explained by the factors we included - dataset choice, k value, training size, and other hyperparameters. The remaining 38% comes from factors we did not measure or random variation.

Precision, recall, and F1 score followed very similar patterns with accuracy. In all four models, the coefficient for implementation type (custom vs sklearn) was not statistically significant ($p > 0.05$), meaning that both implementations produce almost identical predictions. Similarly, apart from hyperparameter choice, dataset choice mattered the most for these metrics.

Predictors of Computational Performance

Producing polar opposite results, implementation type was a highly significant decider for computational metrics. For prediction time, the model achieved an adjusted R^2 of 0.89. Implementation type was the biggest factor where our custom model takes substantially longer than scikit-learn to make predictions.

The number of samples in the dataset was the second biggest factor affecting prediction time (beta = 0.002, $p < 0.001$). More data means more distance calculations, which takes more time. On the other hand, changing k value, distance metric, or voting scheme did not affect prediction time at all. This makes sense, as data size increases the model ends up calculating more distances. The amount of workload directly affects the time it takes to perform those processes.

Fit time showed a small but significant difference between implementations (beta = 0.0001, $p < 0.05$). The effect is tiny because KNN's training phase only stores data, there is no complex learning happening. Due to task simplicity, storing data takes almost no time for either model.

Memory usage showed no significant relationship with implementation type ($p > 0.05$), that is what we expected since both implementations store identical training data. Memory incraesed linearly with training set size (beta = 4.2 bytes per sample, $p < 0.001$) but was not affected by hyperparameter choices.

Summary of Regression Findings

The takeaway is simple: implementation type matters for speed, not accuracy. Our custom model predicts just as well as scikit-learn but takes longer to do it. We assume that the algorithm works as it should while it runs slower due to the lack of optimization similar to the ones used in existing libraries.

Decision Boundary Visualization

Our results so far show that both models perform almost identically in prediction accuracy. Decision boundaries provide a visual confirmation of this finding by showing how each model decides it's prediction while showing points that are the actual real outcomes. We used the Iris dataset for this visualization since its small size produces clear and interpretable boundaries. Both implementations produce identical boundaries across all conditions confirming again the closeness in prediction performance of both models.

Discussion

This following section explains the results we have discovered how they matter.

What the Results Tell Us

We ran 672 experiments across five datasets. Our custom implementation predicts just as well as scikit-learn but runs much slower. Our custom model achieved 92.1% accuracy while scikit-learn achieved 91.5%. The Wilcoxon tests showed statistically significant differences, but we assume the difference will not be significant in real life practice.

The decision boundary plots also confirmed this. Both models draw identical borders across all k values. If our code had bugs or followed the wrong logic, the boundaries would look different. They do not. This gives us confidence that our implementation follows the KNN algorithm correctly.

Why the Speed Gap Exists

Scikit-learn runs 484 times faster than our model. This massive gap is due to how each implementation searches for neighbors. Our model uses a normal KNN theory based approach. For every prediction, it calculates the distance to every single training point, sorts them all, and picks the closest ones. This works, but it is slow.

Scikit-learn uses smarter data structures like KD-trees and ball trees[Pedregosa et al., 2011]. These structures organize training points in a way that lets the algorithm skip large parts of data during the search. Instead of checking every point, it only checks the ones that could possibly be neighbors. On top of that, scikit-learn runs compiled Cython code rather than interpreted Python, which adds another layer of speed[Pedregosa et al., 2011].

The regression analysis confirmed that dataset size is the second biggest factor after implementation type. As data grows, our normal KNN approach suffers more because it has more points to process through. Scikit-learn scales better because its tree structures handle larger data more efficiently.

Hyperparameter Insights

The best k value is between 5 and 10. At k=1, the model relies on a single point which could be noise. At k=100, the model considers too many neighbors and blurs the differences between classes. We saw this pattern across all datasets and both implementations.

Training size matters as expected. More training data leads to better predictions. This aligns with basic statistical theory. Distance metric and voting scheme made smaller differences. Manhattan was slightly better than Euclidean, and distance-weighted voting was slightly better than uniform. Both implementations showed the same patterns.

Practical Implications

For learning purposes, building a model from scratch is valuable. It forces you to understand every step of the algorithm rather than just calling a function. Our implementation correctly replicates KNN

behavior, which as an educational approach has provided us with deeper fundamental KNN and python coding knowledge.

For production use, optimized libraries are the clear choice. On our largest dataset, our model took 126 seconds while scikit-learn finished in under one second.

Limitations

A few limitations can be noted. First, our memory measurement only captures training data size, not the full memory used during prediction. Second, we tested only five datasets which may not cover all possible data types. Third, due to time constraints, our implementation includes only basic KNN features without exploring more advanced options.

Conclusion

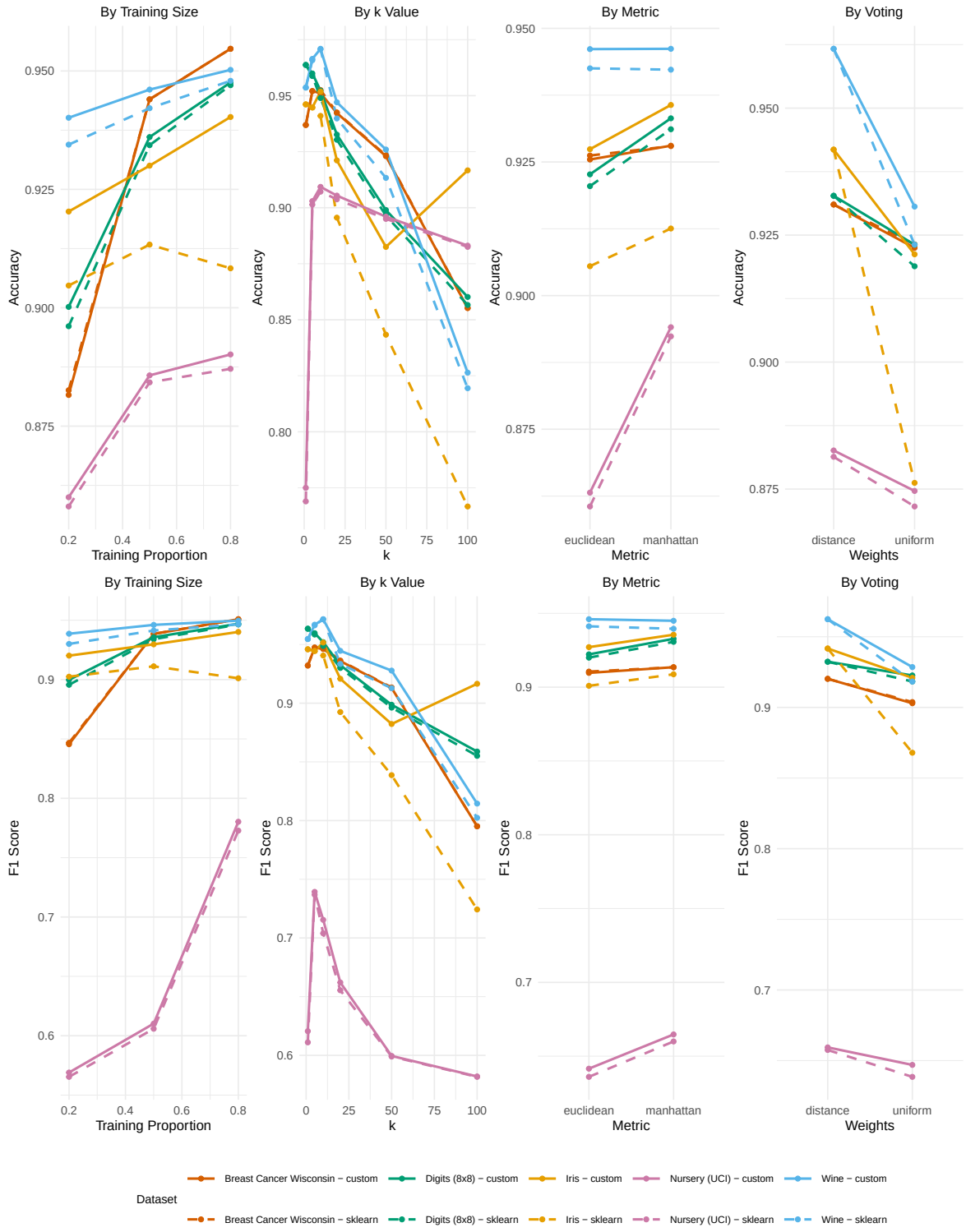
This project implemented a K-Nearest Neighbours classifier from scratch and compared it against scikit-learn across 672 experimental configurations. Through detailed plots, regression analysis and decision boundary visualizations, we found what drives performance in each model and how they differ from each other.

Results show that both implementations produce equivalent predictions with nearly identical accuracy. However, scikit-learn runs significantly faster due to its optimized data structures. Implementation type is not a strong predictor of machine learning performance but it is a strong influencer of computational efficiency.

Building machine learning algorithms from scratch provides valuable educational insight and can achieve good levels of prediction as our implementation clearly shows. But for overall higher performance, especially better computational effectiveness, going with an existing implementation is our suggested choice.

Appendix

Figures 1-2



Figures 3-4

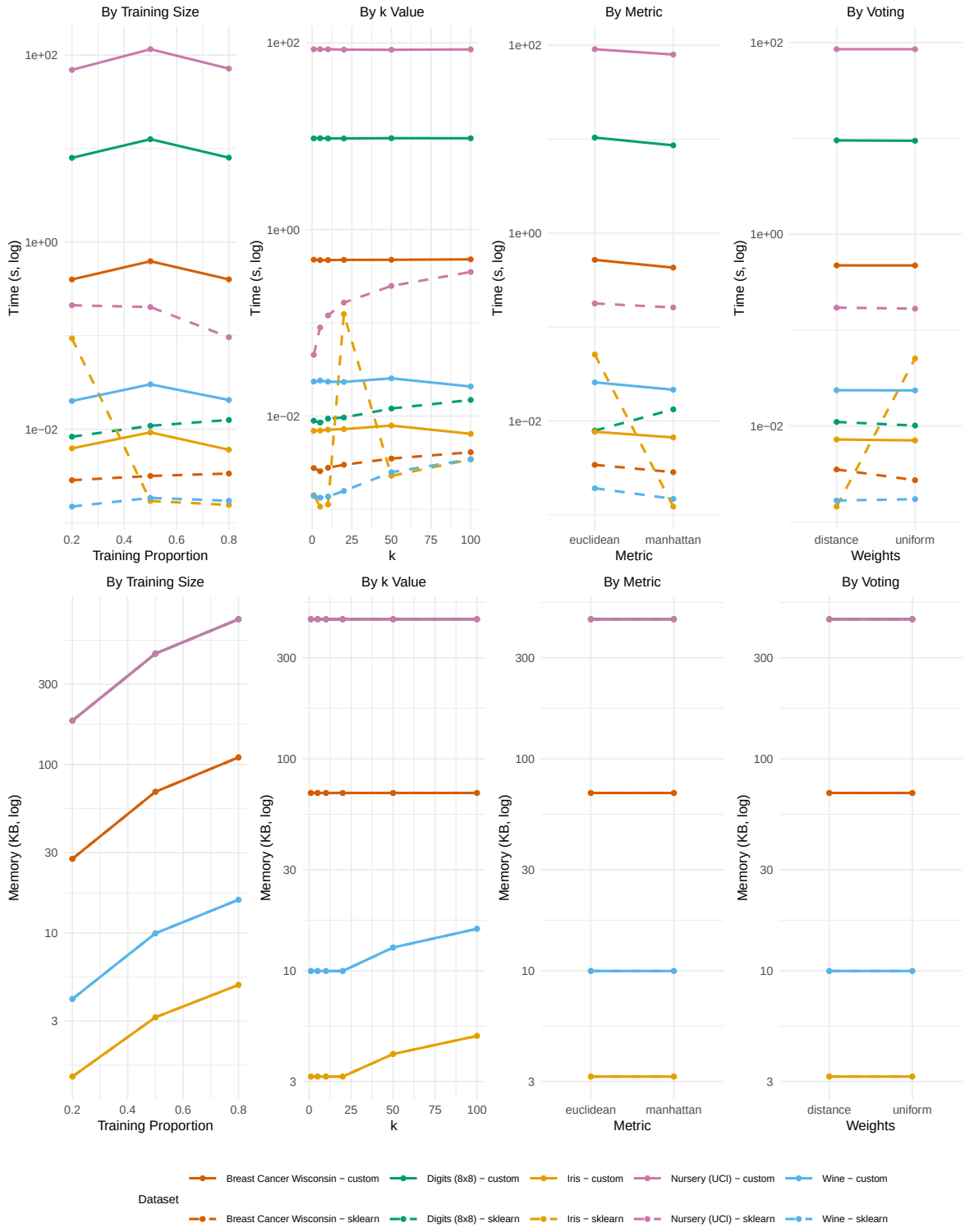
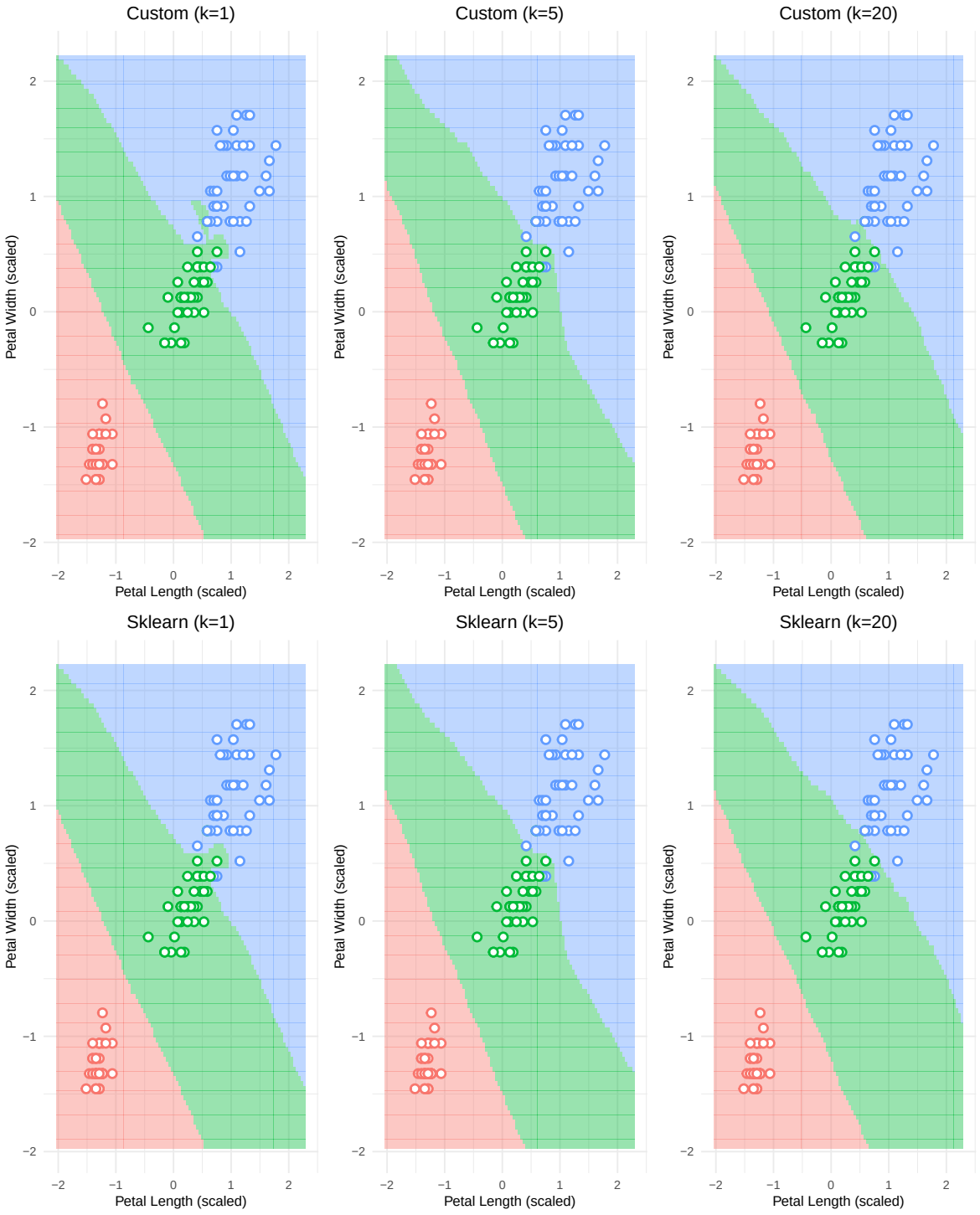

By Training Size
By k Value
By Metric
By Voting
Time (s, log)
1e+02
1e+00
1e−02
Training Proportion
k
Metric
Weights
euclidean
manhattan
distance
uniform
Memory (KB, log)
300
100
30
10
3
Dataset
Breast Cancer Wisconsin − custom
Digits (8x8) − custom
Iris − custom
Nursery (UCI) − custom
Wine − custom
Breast Cancer Wisconsin − sklearn
Digits (8x8) − sklearn
Iris − sklearn
Nursery (UCI) − sklearn
Wine − sklearn

Figure 5: Decision Boundaries − Custom vs Sklearn

Top: Custom Implementation | Bottom: Sklearn | Iris Dataset (Euclidean, Uniform)



References

Haneen Arafat Abu Alfeilat, Ahmad B A Hassanat, Omar Lasassmeh, Ahmad S Tarawneh, Mahmoud B Alhasanat, Hamzeh S Eyal Salman, and V B Surya Prasath. Evaluation of k-nearest neighbour classifier performance for heterogeneous data sets. *SN Applied Sciences*, 1(12):1559, 2019. doi: 10.1007/s42452-019-1356-9. URL https://link.springer.com/article/10.1007/s42452-019-1356-9.

Stefan Aeberhard and M. Forina. Wine. UCI Machine Learning Repository, 1992. URL https://archive.ics.uci.edu/dataset/109/wine.

E. Alpaydin and C. Kaynak. Optical recognition of handwritten digits. UCI Machine Learning Repository, 1998. URL https://archive.ics.uci.edu/dataset/80/optical+recognition+of+handwritten+digits.

Kittipong Chomboon, Pasapitch Chujai, Pongsakorn Teerarassammee, Kittisak Kerdprasop, and Nittaya Kerdprasop. An empirical study of distance metrics for k-nearest neighbor algorithm. In *Proceedings of the 3rd International Conference on Industrial Application Engineering*, pages 280–285, 2015. doi: 10.12792/iciae2015.051.

Pádraig Cunningham and Sarah Jane Delany. k-nearest neighbour classifiers: A tutorial. *ACM Computing Surveys*, 54(6):1–25, 2021. doi: 10.1145/3459665. URL https://dl.acm.org/doi/10.1145/3459665.

Janez Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7:1–30, 2006. URL https://www.jmlr.org/papers/volume7/demsar06a/demsar06a.pdf.

Sahibsingh A Dudani. The distance-weighted k-nearest-neighbor rule. *IEEE Transactions on Systems, Man, and Cybernetics*, SMC-6(4):325–327, 1976. doi: 10.1109/TSMC.1976.5408784. URL https://ieeexplore.ieee.org/document/4309523.

Rezvan Ehsani and Finn Drabløs. Robust distance measures for kNN classification of cancer data. *Cancer Informatics*, 19:1176935120965542, 2020. doi: 10.1177/1176935120965542. URL https://journals.sagepub.com/doi/full/10.1177/1176935120965542.

Andy Field, Jeremy Miles, and Zoë Field. *Discovering Statistics Using R*. Sage Publications, 2012.

Jianping Gou, Taisong Xiong, and Yin Kuang. A novel weighted voting for k-nearest neighbor rule. *Journal of Computers*, 6(5):833–840, 2011. doi: 10.4304/jcp.6.5.833-840. URL https://www.researchgate.net/publication/220405196.

Rajib Kumar Halder, Mohammed Nasir Uddin, Md Ashraf Uddin, Sunil Aryal, and Ansam Khraisat. Enhancing k-nearest neighbor algorithm: a comprehensive review and performance analysis of modifications. *Journal of Big Data*, 11:113, 2024. doi: 10.1186/s40537-024-00973-y. URL https://link.springer.com/article/10.1186/s40537-024-00973-y.

David F Hamilton, Michelle Ghert, and A Hamish RW Simpson. Interpreting regression models in clinical outcome studies. *Bone & Joint Research*, 4(9):152–153, 2015. doi: 10.1302/2046-3758.49.2000571.

Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, 2nd edition, 2009.

Li-Yu Hu, Min-Wei Huang, Shih-Wen Ke, and Chih-Fong Tsai. The distance function effect on k-nearest neighbor classification for medical datasets. *SpringerPlus*, 5(1):1304, 2016. doi: 10.1186/s40064-016-2941-7. URL https://pmc.ncbi.nlm.nih.gov/articles/PMC4978658/.

IBM. What is the k-nearest neighbors algorithm? https://www.ibm.com/think/topics/knn, 2024.

Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An Introduction to Statistical Learning with Applications in R*. Springer, 2013.

Martin Krzywinski and Naomi Altman. Multiple linear regression. *Nature Methods*, 12(12):1103–1104, 2015. doi: 10.1038/nmeth.3665.

Nahomi Lopes and Bernardete Ribeiro. kNN classification: a review. *Annals of Mathematics and Artificial Intelligence*, 91:791–829, 2023. doi: 10.1007/s10472-023-09882-x. URL https://link.springer.com/article/10.1007/s10472-023-09882-x.

James ES Macleod, Andrew Luk, and D Michael Titterington. A re-examination of the distance-weighted k-nearest neighbor classification rule. *IEEE Transactions on Systems, Man, and Cybernetics*, 17(4):689–696, 1987. doi: 10.1109/TSMC.1987.289362. URL https://ieeexplore.ieee.org/document/4309062.

J Arturo Olvera-López, J Ariel Carrasco-Ochoa, and J Francisco Martínez-Trinidad. Decision boundary preserving prototype selection for nearest neighbor classification. *International Journal of Pattern Recognition and Artificial Intelligence*, 19(06):787–806, 2005. doi: 10.1142/S0218001405004332.

Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Edouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. URL https://www.jmlr.org/papers/volume12/pedregosa11a/pedregosa11a.pdf.

David M. W. Powers. Evaluation: From precision, recall and f-measure to roc, informedness, markedness and correlation. *Journal of Machine Learning Technologies*, 2(1):37–63, 2011. URL https://www.researchgate.net/publication/276412348_Evaluation_From_precision_recall_and_F-measure_to_ROC_informedness_markedness_correlation.

pytest Development Team. pytest: helps you write better programs, 2024. URL https://docs.pytest.org/en/stable/getting-started.html.

Oona Rainio, Jarmo Teuho, and Riku Klén. Evaluation metrics and statistical tests for machine learning. *Scientific Reports*, 14(1):6086, 2024. doi: 10.1038/s41598-024-56706-x. URL https://www.nature.com/articles/s41598-024-56706-x.

V. Rajkovic. Nursery. UCI Machine Learning Repository, 1989. URL https://archive.ics.uci.edu/dataset/76/nursery.

Richard J Samworth. Optimal weighted nearest neighbour classifiers. *The Annals of Statistics*, 40(5):2733–2763, 2012. doi: 10.1214/12-AOS1049. URL https://arxiv.org/abs/1101.5783.

Scikit-learn Developers. Nearest neighbors — scikit-learn documentation. https://scikit-learn.org/stable/modules/neighbors.html, 2024.

Gwowen Shieh. Improving on adjusted r-squared. *Collabra: Psychology*, 6(1):45, 2020. doi: 10.1525/collabra.343.

Aakash Singh, Malka N Halgamuge, and Ravishankar Lakshmiganthan. Effects of distance measure choice on KNN classifier performance - a review. *arXiv preprint arXiv:1708.04321*, 2017. URL https://arxiv.org/abs/1708.04321.

S. Sivakumar, Lazaro S. Videla, A. N. Balaji, and B. Sathiyabhama. A simplified approach for efficiency analysis of machine learning algorithms. *PeerJ Computer Science*, 10:e2418, 2024. doi: 10.7717/peerj-cs.2418. URL https://peerj.com/articles/cs-2418/.

Marina Sokolova and Guy Lapalme. A systematic analysis of performance measures for classification tasks. *Information Processing & Management*, 45(4):427–437, 2009. doi: 10.1016/j.ipm.2009.03.002. URL https://www.sciencedirect.com/science/article/abs/pii/S0306457309000259.

UCI Machine Learning Repository. Iris data set. https://archive.ics.uci.edu/dataset/53/iris, 1988.

UCI Machine Learning Repository. Uci machine learning repository. https://archive.ics.uci.edu/, 2024.

Shahadat Uddin, Ibtisham Haque, Haoming Lu, Mohammad Ali Moni, and Ergun Gide. Comparative performance analysis of k-nearest neighbour (knn) algorithm and its different variants for disease prediction. *Scientific Reports*, 12:6256, 2022. doi: 10.1038/s41598-022-10358-x.

Gülden Kaya Uyanık and Neşe Güler. A study on multiple linear regression analysis. *Procedia - Social and Behavioral Sciences*, 106:234–240, 2013. doi: 10.1016/j.sbspro.2013.12.027.

Kilian Q Weinberger and Lawrence K Saul. Distance metric learning for large margin nearest neighbor classification. *Journal of Machine Learning Research*, 10:207–244, 2009. URL https://jmlr.csail.mit.edu/papers/volume10/weinberger09a/weinberger09a.pdf.

Hadley Wickham, Mara Averick, Jennifer Bryan, Winston Chang, Lucy D'Agostino McGowan, Romain François, Garrett Grolemund, Alex Hayes, Lionel Henry, Jim Hester, Max Kuhn, Thomas Lin Pedersen, Evan Miller, Stephan Milton Bache, Kirill Müller, Jeroen Ooms, David Robinson, Dana Paige Seidel, Vitalie Spinu, Kohske Takahashi, Davis Vaughan, Claus Wilke, Kara Woo, and Hiroaki Yutani. Welcome to the tidyverse. *Journal of Open Source Software*, 4(43):1686, 2019. doi: 10.21105/joss.01686. URL https://www.tidyverse.org/.

William Wolberg, Olvi Mangasarian, Nick Street, and W. Street. Breast cancer wisconsin (diagnostic). UCI Machine Learning Repository, 1993. URL https://archive.ics.uci.edu/dataset/17/breast+cancer+wisconsin+diagnostic.

Shichao Zhang, Xuelong Li, Ming Zong, Xiaofeng Zhu, and Ruili Wang. Introduction to machine learning: K-nearest neighbors. *Annals of Data Science*, 3(2):179–206, 2016.